OAUTH OPENS THE DOOR; POLICY DECIDES WHAT YOU SEE

When Kate connects BI Explorer to ACME using OAuth, she grants the scope `collab:doc.read`. This scope allows BI Explorer to access the document-read endpoints in the Customer Collaboration API. From OAuth's point of view, the request is valid: the token is authentic, unexpired, and has the correct scope.

But OAuth scopes only define the outer perimeter of access. When BI Explorer requests documents, the API gateway still checks Cedar to determine what Kate is explicitly allowed to see. For most documents in Kate's workspaces, Cedar responds with allow. When BI Explorer tries to access a document tagged `sensitivity="Legal"`, Cedar responds with `deny`, and that document is blocked, even though the OAuth token is valid.

The policy that enforces this rule is simple:

```
permit(
  principal in ACME::User,
  action == Action::"collab:doc.read",
  resource in ACME::Document
)
unless { resource.sensitivity == "Legal" };
```

This layered approach allows OAuth to focus on what it does best—delegating access to APIs—while Cedar enforces ACME's fine-grained business rules. The result is flexible integration without turning every OAuth token into an all-access pass.

11.2.2 OAuth scopes

Scopes are the mechanism OAuth uses to express those broad contours of access. Instead of granting a client full control over an account, the authorization server issues tokens with specific scopes that specify which parts of an API the client can access. Scopes are strings created by the API provider that represent bundles of related permissions. When a client